

PAACE: 面向计划的自动化智能体上下文工程框架

PAACE: A Plan-Aware Automated Agent Context Engineering Framework

Kamer Ali Yuksel

aiXplain Inc, San Jose, USA

kamer@aixplain.com

摘要

大语言模型 (LLM) 智能体正日益被部署到复杂的多步骤 workflows 中, 这些 workflows 涉及规划、工具使用、反思以及外部知识系统的交互。此类 workflows 会产生迅速膨胀的上下文, 必须对其进行整理、变换与压缩, 才能保持保真度、避免注意力稀释并降低推理成本。以往的摘要与查询感知压缩工作在很大程度上忽略了智能体推理所具有的多步骤、计划感知特性。本文提出 **PAACE (Plan-Aware Automated Context Engineering)**, 面向计划的自动化上下文工程, 这是一个统一框架, 通过未来 k 个任务的相关性建模、计划结构分析、指令协同精炼以及功能保持压缩, 优化 LLM 智能体不断演化的状态。PAACE 包含两部分: (1) **PAACE-Syn**, 一种大规模合成智能体 workflow 生成器, 为 workflow 标注逐步压缩监督; (2) **PAACE-FT**, 一族从成功教师示范中训练得到的蒸馏式计划感知压缩器。在长时程基准 AppWorld、OfficeBench 和 8-Objective QA 上的实验表明, PAACE 在大幅减少上下文负载的同时, 始终能够提高智能体正确性。在 AppWorld 上, PAACE 的准确率高于所有基线, 同时降低了峰值上下文和累积依赖量。在 OfficeBench 与多跳问答上, PAACE 同时提升准确率与 F1, 并减少步骤数、峰值 token 数和注意力依赖量。蒸馏后的 PAACE-FT 保留了教师模型 97% 的性能, 同时将推理成本降低一个数量级以上, 使紧凑模型能够实际部署计划感知压缩。

1 引言

LLM 驱动的智能体已经成为解决复杂多步骤任务的核心范式, 应用领域包括软件开发、研究辅助、运营自动化、法律工作流、数据分析和企业决策。ReAct (Yao et al., 2023a)、Toolformer (Schick et al., 2023a)、AutoGPT、Devin、WebArena (Zhou et al., 2024) 与 AgentBench (Liu et al., 2023) 等系统, 既展现了构建具有推理、规划和工具使用能力的智能体的前景, 也揭示了其中的挑战。随着这些系统演进, 一个持续存在的瓶颈愈发明显: 上下文管理。LLM 智能体的状态并不由模型参数表示, 而是由其提示上下文表示, 其中包括系统指令、持续演化的计划、先前推理轨迹、工具结果、用户指令、长期记忆、检索到

的知识和中间输出。智能体并非以单步方式运行。

它们执行任务之间存在依赖关系的计划 $\Pi = [\tau_1, \tau_2, \dots, \tau_n]$, 而每个阶段只与上下文的一部分相关。随着任务深度与广度增长, 这一状态会变得越来越庞大、嘈杂、冗余, 处理成本也越来越高。即使模型具有 20 万至 100 万 token 的窗口, 当无关或结构不良的信息过载时, 推理质量仍会下降, 即所谓“上下文腐化”(context rot)。近期行业报告强调, 现代智能体的失败绝大多数是上下文失败, 而非模型失败。尽管模型架构与上下文长度已经进步, 智能体仍会因以下情况失败: 关键信息被丢弃或埋没在无关文本中; 无关细节使模型注意力过载; 多个任务争夺上下文带宽; 指令随时间互相矛盾或发生漂移; 上下文过长而无法高效处理。我们将上下文工程表述为: 针对智能体不断演化的执行状态, 学习一种由结果保持监督训练得到的状态压缩策略, 而不是进行启发式提示编辑。

提示工程优化的是初始指令, RAG 系统优化的是检索; 而缺失的学科是上下文工程, 即持续优化智能体在每一步所见内容的科学。现有方法只是部分处理了这一问题。BART (Lewis et al., 2019)、FLAN-T5 (Chung et al., 2022) 等传统摘要器与指令遵循摘要器, 以及 LLMLingua (Jiang et al., 2023) 等面向压缩的方法, 能够生成简洁摘要, 却经常移除多步骤推理所需的结构依赖。摘要会抹平步骤之间的因果联系, 从而损害智能体规划和工具使用 workflow。Self-RAG (Asai et al., 2024) 与 LLMLingua-2 (Jiang et al., 2024) 等方法针对单个即将到来的查询优化相关性, 但它们不建模未来 k 个步骤、多跳依赖或持续演化的计划。Provence (Wu et al., 2024) 使用相关性分类器进行二元保留/丢弃式裁剪, 却不支持重写、指令精炼、依赖跟踪或结构化上下文塑形。

现代长上下文 LLM (Claude 3.5 Sonnet、Gemini 1.5 Pro、GPT-4o mini L) 提供 20 万至 100 万以上 token 的窗口, 但在实践中仍受到注意力稀释、上下文“腐化”和二次方成本增长的影响 (Zaheer et al., 2020; Guo et al., 2021)。大窗口无法解决上下文结构不良或无关的问题。MemAgent (Yu et al., 2025a)、Reflexion (Shinn et al., 2023) 和 Generative Agents (Park et al., 2023) 等记忆与反思系统改善了检索和情景记忆, 却不执行上下文塑形、状态重构或下一条指令精炼。Yu et al. (2025b) 通过自然语言压缩准则优化下一

步相关性，但它不建模未来 k 步计划结构、不精炼指令，也无法联合优化计划感知的上下文与指令变换。我们的首要区别在于以多个未来计划步骤和全局 workflow 结构作为条件。现有系统均未同时考虑：

- 计划结构（前置条件、中间状态、时间依赖）与未来 k 个任务的相关性；
- 指令精炼与上下文选择的联合进行；
- 面向多步骤 workflow 的全局优化；
- 用于上下文塑形的可学习策略。

贡献

1. **PAACE**：一个用于计划感知上下文工程的框架，统一上下文裁剪、摘要、重写、压缩、重新注入和任务精炼。
2. 一种上下文相关性形式化：纳入计划依赖、未来 k 个任务的相关性、结构分解以及指令与上下文的协同演化。
3. 显式相关性评分信号：用于选择与优化，包括计划结构、未来 k 个任务的保持、结果保真度和信息论意义上的保留。
4. **PAACE-Syn**：一种可扩展的合成数据生成系统，可产生数百万条带标注的工作流轨迹。
5. **PAACE-FT**：一族专门面向上下文塑形与压缩的微调 LLM。
6. 全面实验：表明 PAACE 在多个智能体基准上改善了正确性、效率和成本。

这些贡献共同确立了计划感知上下文工程在稳健长时程智能体设计中的关键地位，并提供首个联合建模计划相关性与上下文结构的端到端框架，从而支持成本高效且高保真的智能体推理。

2 相关工作

本节综合讨论与 PAACE 相关的十个研究领域：（1）摘要与压缩；（2）查询感知和任务感知的缩减；（3）长上下文模型；（4）智能体记忆架构；（5）检索增强智能体；（6）智能体规划与多步骤推理；（7）提示和指令优化；（8）上下文裁剪与选择；（9）多智能体系统与元推理；（10）启发人工记忆系统的认知框架。尽管每个领域都为上下文管理贡献了技术，但此前没有工作提供类似本文方法这样统一、计划感知且面向未来 k 个任务的上下文工程框架。

文本摘要是最早的上下文缩减形式之一。Pointer-Generator (See et al., 2017)、PEGASUS (Zhang et al., 2019)、BART (Lewis et al., 2019) 和 T5 (Raffel et al., 2020) 等经典抽象式摘要模型能够对文档进行高层压缩，却不具备任

务感知或计划感知能力。FLAN-T5 (Chung et al., 2022) 等指令遵循摘要器和基于 LLM 的摘要器（如 GPT-4、Claude 3）更加稳健，但仍难以保留多步骤智能体 workflow 所需的功能依赖。LLMLingua (Jiang et al., 2023) 和 LLMLingua-2 (Jiang et al., 2024) 使用学习得到的保留模型进行 token 级压缩，在任务性能仅轻微下降的情况下大幅缩短上下文。然而，LLMLingua 是查询感知而非计划感知的，其优化目标是单轮问答相关性，而不是多步骤智能体任务。Big-Bird (Zaheer et al., 2020)、LongT5 (Guo et al., 2021) 和层次化 Transformer 等其他方法扩展了上下文窗口，却不优化上下文的内容。PAACE 不止进行摘要，还纳入计划结构、任务依赖、指令精炼，以及从合成 workflow 中学习的相关性信号。

近期模型尝试为单个即将到来的任务优化上下文。Self-RAG (Asai et al., 2024) 根据问题生成检索查询，并在将上下文传给 LLM 前裁去无关内容。Self-RAG 虽然对问答很有效，却不够通用于多步骤智能体 workflow，因为后者需要建模未来任务、工具输出和计划转换。类似地，ACon (Agent Context Optimization) (Yu et al., 2025b) 通过自然语言反馈优化“压缩准则”，以压缩智能体交互历史。ACon 改善了短时程任务的上下文相关性，但它：（1）不建模计划结构；（2）局限于下一步压缩；（3）缺少指令精炼；（4）不联合优化计划感知的上下文选择。首要区别在于，PAACE 显式地以多个未来计划步骤和全局 workflow 结构作为压缩条件，因此将 ACon 泛化为一个多任务、多步骤的结构化框架。Provenance (Wu et al., 2024) 引入二元相关性分类器来裁剪检索文档片段。这改善了 RAG 流水线，但对智能体 workflow 过于粗粒度，因为推理链、状态表示和计划需要更细粒度的变换。

许多模型试图通过增大上下文窗口来处理长时程推理，包括 GPT-4o mini (128k)、Claude 3.5 Sonnet (200k–1M)、Gemini 1.5 Pro (1M) 和 DeepSeek-V2 (256k+)。然而，长上下文窗口无法解决相关性稀释问题：在大型输入上注意力会退化，导致上下文“腐化”和推理失败。另一些方法则引入显式的上下文或记忆控制器。MemAgent (Yu et al., 2025a) 使用强化学习覆写固定大小的记忆状态，从而扩展到数百万 token，但其目标是文档问答而非智能体 workflow，既不精炼指令，也不建模计划结构。Compressive Transformers (Rae et al., 2020) 及相关架构方法在模型内部执行学习式压缩，却缺乏任务或计划层面的落地。较新的 COMPASS (Li et al., 2024)、FoldGRPO (Sun et al., 2025)、ReCAP (Zhang et al., 2025) 和 GoA (Joo et al., 2025) 等系统，通过学习式控制器、强化学习或免训练的多智能体协调来探索上下文控制；它们通常操作固定大小的潜在记忆，或要求修改架构或推理过程。这些方法与 PAACE 互补：它们修改模型架构或记忆机制，而 PAACE 纯粹在上下文层面运行，可与任意智能体或骨干模型集成。任务无关的学习式压缩器 (Sun et al., 2023) 在不以未来计划结构

为条件的情况下优化通用提示缩减；与之相反，PAACE 显式建模智能体工作流程中未来 k 个任务的依赖。不同于启发式提示压缩，PAACE 从结果层监督中学习以潜在智能体状态为对象、以计划为条件的策略，将自身定位为一种经验型系统机器学习方法，其目标位于轨迹层面而非 token 层面。

2.1 LLM 智能体的记忆架构

人类记忆由工作记忆、情景记忆、语义记忆和程序性记忆构成。AI 研究者已使用这一分类法设计智能体记忆栈 (Park et al., 2023; Shinn et al., 2023)。PAACE 将上下文分解为可行动、结构性、指代性和论证性成分，这与上述认知结构相呼应，并为上下文相关性提供扎实的理论基础。若干系统受到人类记忆的启发：

- **Reflexion** (Shinn et al., 2023) 存储智能体的自我评估，并检索这些评估以进行自我改进；
- **Generative Agents** (Park et al., 2023) 维护情景记忆并综合高层反思，从而驱动可信行为；
- **LTM/RAG** 系统为智能体集成持久化向量记忆；
- 层次化记忆（短期、长期与全局记忆）正越来越多地用于 AI 智能体。

这些系统强调存储、检索和记忆蒸馏，却不优化推理期间呈现给 LLM 的上下文形态、结构或内容。PAACE 在这些系统之上集成结构化重写、任务依赖压缩和指令精炼。PAACE 还通过任务条件化的裁剪、重写和指令改进，提高工具调用的可靠性。Toolformer (Schick et al., 2023b)、ReAct (Yao et al., 2023b)、MRKL (Ahn et al., 2022) 和 PAL (Gao et al., 2023) 等工具使用框架，使 LLM 能将推理扎根于外部工具。复杂的智能体框架会结合检索、规划和工具执行，但仍存在以下问题：

- 检索相关性通常局限于单个即将到来的查询；
- 检索到的内容只是被追加，而未被塑形或压缩；
- 工具输出不断累积，很少得到蒸馏。

CoT (Wei et al., 2022)、ToT (Yao et al., 2023c)、RAT (Zhang et al., 2023) 和 RCT (Zhou et al., 2023c) 等多步骤推理框架提供规划机制，却不管理累积的上下文。Planner-LM (Huang et al., 2024) 探索显式规划模块，却不优化计划与上下文之间的交互。PAACE 则将上下文视为规划所依赖的状态，并建模上下文各组成部分对未来任务的相关性。提示优化与进化式提示繁育 (Zhou et al., 2023a; Pryzant et al., 2023) 使用反馈循环生成改进后的指令。类似地，DPO (Rafailov et al., 2023)、RLHF (Ouyang et al., 2022) 和 PRO (Saunders et al., 2024) 对指令或偏好进行精炼。然而，它们均未涉及面向未来 k 个任务的指令精炼、计划条件化的指令重写、指令与上下文的协同精炼，或智能体工作流程执行期间的指令更新。据我们所知，PAACE 首次提出包含指令在内的完整上下文工程系统方法。

除了摘要之外，Wu et al. (2024) 等裁剪框架旨在移除无关文本。仅靠裁剪不足以服务多步骤智能体，因为一些内容应被重写而非删除。任务依赖是多跳的，因此必须保留全局计划结构；如果不与上下文同步重写，指令也会漂移。PAACE 将裁剪作为更大的结构化上下文变换算子库中的一种算子。Reflexion (Shinn et al., 2023)、AlphaAI 系统、CrewAI 和 LATS (Zhou et al., 2023b) 等元智能体使用多智能体循环稳定推理，一些系统还包括元级评估或自我批判。然而，它们均未提出专门负责上下文工程的元智能体、微调专用模型 (PAACE-FT)，或生成计划条件化合成数据集 (PAACE-Syn)。我们将上下文工程引入为元智能体的核心功能。此前没有统一框架能够联合执行：计划感知且面向未来 k 个任务的上下文优化、指令精炼、重写与裁剪与摘要与压缩、对 workflows 的结构化推理，以及专门面向上下文工程的模型训练。据我们所知，PAACE 是首个共同处理上述问题的框架。

3 方法

PAACE 通过两阶段范式优化多步骤 LLM 智能体不断演化的上下文：(1) 一个教师 LLM，在持续演化的自然语言提示引导下执行计划感知且面向未来 k 个任务的压缩；(2) 一个蒸馏后的学生模型，以低成本模仿教师的成功压缩。与手工设计上下文编辑算子的以往工作不同，PAACE 以数据驱动方式学习完整的压缩策略：教师提示通过 LLM 驱动的进化循环，在真实多步骤 workflow 上进行元学习；学生则完全根据教师的最佳示范训练。PAACE 通过长时程轨迹上的结果层筛选，以经验方式学习该策略，而不是使用可微目标或闭式保证；其优先目标是在真实智能体工作负载中实现稳健的状态保真度与任务成功。该框架包含两个组件：

- **PAACE-Syn**：大规模生成成长时程、含噪声的智能体 workflow，以及成对的完整上下文轨迹和压缩上下文轨迹；
- **PAACE-FT**：将高质量教师压缩蒸馏为专用的计划感知压缩器。

我们将智能体上下文视为潜在执行状态，并学习以计划为条件、保持结果的压缩策略。在步骤 t ，LLM 智能体维护上下文状态

$$C_t = \{I_0, P, \Pi, H_{0:t}, O_{0:t}, R_{0:t}, M\},$$

其中， I_0 是初始用户指令， P 是系统提示， $\Pi = [\tau_1, \dots, \tau_n]$ 是显式或隐式计划， $H_{0:t}$ 是推理轨迹， $O_{0:t}$ 是工具或环境观测， $R_{0:t}$ 是检索到的文档， M 是长期记忆。PAACE 不使用手工算子显式操纵这些组成部分；相反，它们隐式影响教师（随后也影响学生）学会保留或丢弃的内容。

步骤 t 的压缩定义为映射

$$\tilde{C}_t = \text{TeacherCompress}(C_t, \Pi_{t:t+k}; p), \quad (1)$$

其中, $\Pi_{t:t+k}$ 表示剩余工作流 DAG 子路径上接下来的 k 个步骤 (k 随任务而定), p 是自然语言压缩提示。教师以接下来的任务或完整工作流描述为条件, 因此能够保留多个后续步骤所需的信息。PAACE 不假设闭式参数化相关性模型, 也不使用 token 级可微损失。相反, 计划感知相关性通过在轨迹层面评估的显式、可计算信号来操作化: (i) 完整上下文结果与压缩上下文结果之间的语义等价性; (ii) 逐步压缩率; (iii) 通过条件化而隐式覆盖未来 k 个计划步骤; (iv) 来自学习式 LLM 评估器的偏好判断。这些信号共同定义一个面向任务保真度与上下文效率的不可微目标; 该目标通过完整智能体 rollout 上的黑盒选择来优化, 而不是通过 token 级损失来优化。

尽管 PAACE 的动机来自任务图、因果影响和信息保留等概念, 我们在当前实现中并未实例化显式符号任务图或闭式信息论目标。相反, 这些概念通过以下方式隐式捕获: 让压缩以智能体工作流图 (DAG) 的部分剩余子路径为条件, 并依据完整上下文执行结果, 使用显式、可测试的标准进行结果层选择。因此, 本文所说的“形式化”是指可操作且可验证的选择机制, 而非解析推导或符号保证。计划描述 Π 或者直接取自提供结构化任务分解的基准 (如 AppWorld、OfficeBench), 或者由使用与执行相同骨干模型的 LLM 规划器生成。PAACE 不要求计划完美: 当计划步骤缺失或部分错误时, 未来 k 步条件化会使压缩质量平缓下降, 却不会对执行造成灾难性影响。在实践中, 它更像柔性的计划感知正则化器, 而非脆弱的计划依赖控制器。

3.1 合成上下文数据集生成

PAACE 构建大规模长时程智能体工作流合成语料库。每个工作流 W 采样为

$$W = (I_0, \Pi, \{\tau_i\}_{i=1}^n, \phi),$$

其中, I_0 是包含无关日志与不完整摘要的冗长、含噪初始输入; Π 是计划的自然语言描述; τ_i 是任务指令; ϕ 是最终输出要求。PAACE-Syn 涵盖的任务领域包括: 以文档为中心的工作流、网页导航与多应用交互, 以及带检索的多跳问答。计划长度为 5 至 30 步, 包含搜索和浏览操作、文档和文件操作、电子表格格式表格操作, 以及对检索内容的结构化信息抽取与聚合等工具交互。对每个工作流, 智能体首先在不压缩的情况下运行:

$$C_1^{\text{full}} = \{I_0, P, \Pi\}, \quad (2)$$

$$C_{t+1}^{\text{full}} = \text{Update}(C_t^{\text{full}}, \text{Agent}(C_t^{\text{full}}, \tau_t)), \quad (3)$$

并从 C_{n+1}^{full} 产生最终答案 $\hat{y}^{\text{full}}$ 。随后使用教师压缩重新执行同一工作流。在步骤 t , 给定当前上下文 C_t 和计划切

片 $\Pi_{t:t+k}$, 教师使用提示 p 产生压缩上下文 $\tilde{C}_t$ 。之后, 智能体只使用 $\tilde{C}_t$ 行动:

$$\tilde{C}_t = \text{TeacherCompress}(C_t, \Pi_{t:t+k}; p), \quad (4)$$

$$\tilde{C}_{t+1} = \text{Update}(\tilde{C}_t, \text{Agent}(\tilde{C}_t, \tau_t)). \quad (5)$$

最后一步结束后, 智能体从 $\tilde{C}_{n+1}$ 产生 $\hat{y}^{\text{comp}}$ 。对每一步 t , PAACE 记录元组

$$(\Pi_{t:t+k}, C_t, \tilde{C}_t)$$

以及压缩统计量。PAACE 使用若干指标, 将每条压缩轨迹与其完整上下文对应轨迹进行比较。令 $\text{embed}(\cdot)$ 为固定的句子嵌入模型, 我们按下式计算语义等价性:

$$s = \cos(\text{embed}(\hat{y}^{\text{full}}), \text{embed}(\hat{y}^{\text{comp}})). \quad (6)$$

同时, 将工作流描述以及输出 $\hat{y}^{\text{full}}$ 和 $\hat{y}^{\text{comp}}$ 一同提供给评估 LLM; 它返回一个二元标签 (更好与更差/相同) 以及简短理由。这构成对嵌入相似度的补充稳健性检查。对每一步 t , 我们计算基于字符或 token 的压缩率 $r_t = |\tilde{C}_t|/|C_t|$ 。我们要求 $0 < r_t < 1$, 以避免空输出或比原文更长的退化压缩。只有成功轨迹才被用作学生模型的监督。当且仅当满足以下条件时, 一条压缩轨迹被标记为成功:

- 语义等价性超过阈值: $s \geq \theta$ (通常 $\theta = 0.85$);
- 各步骤满足 $0 < r_t < 1$, 并产生非空的 $\tilde{C}_t$;
- 判断模型不认为压缩答案严格差于完整答案。

PAACE 只从生成的所有轨迹中抽取成功的压缩样本。对每个此类样本, 我们构造如下监督元组:

$$(\underbrace{\Pi_{t:t+k}}_{\text{后续任务}}, \underbrace{C_t}_{\text{上下文}}) \mapsto \underbrace{\tilde{C}_t}_{\text{压缩上下文}}. \quad (7)$$

学生模型的典型输入将未来 k 步指令与完整上下文拼接起来, 目标则是教师生成的压缩上下文。由于只保留语义等价性高且压缩有效的轨迹, 所得数据集包含丰富的、针对计划结构与长时程依赖定制的功能保持压缩。

3.2 教师压缩蒸馏

PAACE-FT 训练专用 LLM 来近似教师面向未来 k 个任务的压缩映射。令 $x = (\Pi_{t:t+k}, C_t)$ 表示输入文本, $y = \tilde{C}_t$ 表示教师压缩后的上下文。学生模型使用标准因果语言建模损失训练:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(x,y)} \sum_{i=1}^{|y|} \log p_{\theta}(y_i | x, y_{<i}), \quad (8)$$

其中, x 的 token 在损失中被掩蔽。蒸馏并非为了超越教师, 而是要用紧凑模型取代昂贵的压缩器, 同时保留计划感知的压缩行为。通过蒸馏, 学生学会:

- 将长上下文压缩为短得多的形式；
- 保留多个未来任务会使用的变量、标识符和约束；
- 丢弃无关日志、噪声和走入死路的推理；
- 在没有显式符号表示的情况下隐式遵循计划结构。

推理时，可以在每一步反复调用学生模型，无须运行进化过程或调用大型教师模型，即可快速产生计划感知压缩。学生行为蒸馏自教师在数千个合成工作流上的表现，因此 PAACE 能使智能体在长时程中维持紧凑、相关的上下文，同时保持正确性、提高对上下文稀释的稳健性并降低推理成本。部署过程如下：

1. 智能体获得或构造计划 $\Pi = [\tau_1, \dots, \tau_n]$ ；
2. 在步骤 t ，给定当前上下文 C_t 和切片 $\Pi_{t:t+k}$ ，学生压缩器产生 $\tilde{C}_t$ 。智能体以 $\tilde{C}_t$ 为条件执行任务 τ_t ，并产生更新后的上下文 C_{t+1} ；
3. 重复步骤 2，直至计划完成并产生最终答案。

教师的压缩行为完全由自然语言提示 p 控制。PAACE 不手工调节 p ，而是将上下文压缩视为黑盒策略，通过 LLM 驱动的进化在多个工作流上优化提示种群，以元学习方式得到提示。种群中的每个提示变体 p_i 都关联有根据评估估计的统计量：

- 成功率：被标记为成功的压缩轨迹所占比例；
- 成功轨迹上的平均语义等价性；
- 成功轨迹上的平均压缩率；
- 在保真度与压缩之间权衡的标量奖励。

PAACE 将奖励、成功率、平均等价性和平均压缩率的排名组合成用于选择的复合分数。进化以稳态、异步方式运行。在任意时刻，每个工作进程被分配一个提示变体，并在若干独立工作流上评估该变体，产生完整上下文轨迹与压缩上下文轨迹、轨迹标签（成功/失败）以及该变体的更新统计量。新结果到达时，提示变体会在线更新，全局奖励统计量也会持续重算。随着新提示根据真实多步骤性能被提出、评估和选择，教师的压缩策略会逐步改进。

3.3 监督质量、适用范围与稳健性

由于监督由 LLM 教师和评估器生成，PAACE-Syn 可能继承教师的压缩偏好。为缓解这一问题，我们强制要求其结果与完整上下文执行严格一致，并丢弃任何降低任务成功率的压缩，即使判断模型偏爱该压缩也不例外。经验上，这种筛选会减少模式坍塌，并防止模型过拟合风格化或启发式压缩伪影。此外，我们强调，PAACE 中的指令协同精炼是隐式的，而非独立的优化目标：只有当指令嵌入压缩上下文并随之塑形时，指令才会被重写。PAACE 不执行显式的纯指令优化；将指令精炼单独作为目标是未来工作。

嵌入相似度与基于 LLM 的判断虽然为任务保真度提

供了可扩展且经验稳健的代理指标，却不能保证形式上的语义等价或安全关键正确性。因此，PAACE 针对的是相对于基准任务结果的功能保持，而非形式化验证。纳入符号检查或领域专用验证器，是未来工作的一个重要方向。尽管存在这些局限，同时使用嵌入相似度与 LLM 判断仍能大幅减少退化压缩。在合成验证工作流上的消融表明，移除任一筛选器都会使失败率增加约 8–12%，主要原因是过度激进地删除信息，或指令漂移虽保留了表面相似性却破坏了下游任务执行。

4 实验

我们在两个需要多步骤推理、工具使用和跨应用状态跟踪的长时程智能体基准上评估 PAACE。这些基准从规划、观测整合、记忆需求和长上下文稳健性等方面提供多样化评估信号。对每个基准与模型设置，我们在固定执行骨干、规划器、工具接口和推理配置下，将 PAACE 与基线比较。同一比较组内，各方法之间唯一的区别是上下文管理策略。需要强调的是，PAACE 不依赖任何任务专用启发式规则。所有压缩均从 PAACE-Syn 生成的合成计划条件化示范中学习，并被蒸馏进 PAACE-FT。教师压缩器使用上下文窗口为 65,536 token 的 OpenAI GPT-OSS-120B 模型实现。该模型可提供未来 k 步压缩所需的稳定多跳推理质量。部署用的学生压缩器 PAACE-FT 被蒸馏至 Qwen3-4B-Instruct，因为它具有足够大的上下文窗口和良好的摘要质量。

为评估不依赖专有 API 的定性可复现性，我们使用开放权重的 GPT-OSS-20B 模型进行了辅助合理性检查。由于其规模小得多，这些运行没有纳入定量比较表；不过，它们确认了计划感知的未来 k 步压缩所产生的定性效应，以及不同方法在各基准上的相对排序都能保持。

PAACE-Syn 约含 120 万个合成工作流，压缩前合计约 95 亿 token；这些数据完全离线生成，其成本在基准内所有下游任务之间分摊。我们计划在论文发表时发布完整代码库、提示和合成数据集的一个代表性子集，以支持可复现性与后续研究。使用 PAACE-Syn 产生的合成监督，蒸馏压缩器 PAACE-FT 在将推理成本降低一个数量级以上的同时，保留了教师模型 97–98% 的性能。将 PAACE-FT 接入智能体循环后，它获得了与教师压缩版本近乎相同的准确率，确认未来 k 步压缩行为可以良好迁移到小模型。教师压缩器开销很大，仅用于离线数据生成。蒸馏完成后，PAACE-FT 不再产生优化或教师调用成本，并可在同一环境的不同任务之间复用，无须额外的提示进化或 rollout。推理时，PAACE-FT 每步增加的延迟低于 8%，同时将总输入 token 减少 35–60%，最终降低智能体成本。

实验的主要目标是评估：

- (RQ1) 在长上下文压力下，计划感知的未来 k 步压缩

能否提高智能体正确性；

- **(RQ2)** PAACE 能否在保持任务保真度的同时减少峰值上下文长度；
- **(RQ3)** 教师-学生 PAACE-FT 模型能否保持压缩质量。

PAACE 不会把裁剪、重写和摘要显式分解为独立参数化的算子；相反，这些行为由学习到的压缩策略共同涌现。因此，算子级消融在本设定中并无明确含义，因为裁剪、重写和摘要都从同一个学习策略中联合涌现。不过，我们评估了一个受约束的变体：其中禁止指令重写，压缩器只允许执行抽取式删除。这一限制在多步骤任务上导致明显更高的失败率，说明隐式指令塑形对长时程稳健性有实质贡献。我们在三类长时程智能体任务上评估所提方法（PAACE）：

- **AppWorld** (Trivedi et al., 2024)：具有异构观测的多应用任务；
- **OfficeBench** (Wang et al., 2024)：以文档为中心的工具链和结构化操作；
- **Multi-Objective QA** (Zhou et al., 2025)：使用工具搜索的多跳检索问答。

每个基准同时测量任务成功率和依赖交互的效率指标。本文中，效率均指 *token* 效率，即有效上下文长度与累积注意力依赖量的下降：

- **Acc / EM / F1**：由基准定义的任务性能；
- **Peak**：轨迹中的最大输入上下文长度；
- **Dependency**：累积注意力负载，定义为 $\text{Dep} = \sum_{t=1}^T |C_t|$ ，其中 $|C_t|$ 是步骤 t 的输入 *token* 数。该指标以百万 *token* 报告，用于近似 Transformer 的总注意力成本，并与延迟和二次方计算增长相关。由于所有方法都使用相同的智能体骨干和步骤数，即使峰值上下文长度不同，也可以直接比较各种压缩策略的 Dependency。

我们将 PAACE 与 ACon (Yu et al., 2025b) 以及下列上下文缩减基线进行比较：

- **No Compression**：保留完整交互历史；
- **FIFO**：保留最近 k 轮，丢弃更早内容；
- **Retrieval**：基于嵌入选择过往交互；
- **LLMLingua**：抽取式长上下文压缩；
- **Prompting**：使用启发式摘要指令。

表 1、表 2 和表 3 汇总了 AppWorld、OfficeBench 与 8-Objective QA 上的结果。PAACE 在显著降低上下文成本的同时，性能始终不低于或优于全部基线。PAACE 在多个基准上超过无压缩基线，这说明计划感知压缩可以充当一种正则化，减少无关或陈旧上下文造成的干扰。在两个长时程基准上，PAACE 相比所有基线都表现出持续更高的任务性能、显著更低的上下文消耗，以及更稳定的多步骤推理。结果证明，将计划结构、未来 k 步相

表 1: AppWorld 平均结果；标准差反映 Easy、Medium 和 Hard 三种难度。在所有基准上，PAACE 在降低上下文规模和成本的同时保持状态保真度，并提高多跳工作流的准确率。

方法	Acc↑	Steps↓	Peak↓	Dep↓
No Compression	56.00	16.14	9.93	5.96
FIFO	45.80	28.48	6.73	5.69
Retrieval	27.40	33.17	8.39	6.68
LLMLingua	39.30	24.42	7.50	6.37
Prompting	43.50	24.01	6.93	5.29
ACON UT	51.20	20.92	7.17	4.49
ACON UTCO	56.50	22.82	7.33	4.69
PAACE (本文)	59.00	19.20	6.23	3.75
标准差	±9.68	±5.92	±0.89	±1.60

表 2: OfficeBench 上的结果。

方法	Acc↑	Steps↓	Peak↓	Dep↓
No Compression	76.84	11.52	7.27	4.43
FIFO	67.37	12.26	4.02	2.64
Retrieval	65.26	16.20	4.33	2.06
LLMLingua	70.53	10.89	4.65	1.85
Prompting	71.58	10.13	4.40	1.10
ACON UT	74.74	13.13	4.93	3.85
ACON UTCO	72.63	11.54	4.54	1.91
PAACE (本文)	78.10	10.48	4.29	1.64

关性和工作流级合成监督纳入上下文优化十分重要。即使使用紧凑压缩器，PAACE 也能使长时程智能体在不超过实际上下文预算的情况下保持连贯的状态表示。总之，AppWorld、OfficeBench 与 Multi-Objective QA 上的实验结果表明，计划感知上下文工程是稳健长时程智能体的关键组成。PAACE 持续提升准确率、降低上下文成本并稳定多步骤推理，说明上下文优化应被视为核心架构模块，而非辅助工具。我们注意到，相比最强基线 ACON UTCO 的改进相对于观测方差有时较小。因此，这些结果应被解释为：PAACE 在显著降低上下文成本的同时不会损害任务性能，而不应被解释为它在该基准上带来了巨大的绝对准确率增益。表 4 显示，适度前瞻 ($k = 2$) 足以服务工具型基准，而多跳问答受益于更长的相关性视野 ($k = 3$)，因为检索到的证据往往在获取数步之后才被使用。更大的 k 会因压缩难度增加而带来递减收益，这说明未来 k 步条件化应根据任务依赖结构来选择。

一个核心发现是，智能体不仅会受益于保留相关信息，也会受益于移除具有干扰性和已经过时的信息。尽管

表 3: 8-Objective QA 上的平均结果。

方法	EM↑	F1↑	Steps↓	Peak↓	Dep↓
No Compression	0.366	0.488	15.78	10.35	3.32
FIFO	0.293	0.388	19.26	5.09	2.51
Retrieval	0.331	0.438	20.06	5.11	2.62
LLMLingua	0.363	0.481	17.68	5.68	2.24
Prompting	0.376	0.478	18.70	4.73	1.66
ACON UT	0.373	0.494	17.14	4.71	1.57
ACON UTCO	0.335	0.458	17.79	4.65	1.50
PAACE (本文)	0.402	0.512	16.86	4.41	1.41

表 4: PAACE 的未来 k 步条件化消融。

基准	k	Acc / EM↑	F1↑	Peak↓	Dep↓
AppWorld	1	56.5	—	6.05	3.95
	2	59.0	—	6.23	3.75
	3	58.6	—	6.48	3.82
OfficeBench	1	76.3	—	4.15	1.72
	2	78.1	—	4.29	1.64
	3	77.6	—	4.51	1.69
8-Objective QA	1	0.381	0.497	4.18	1.36
	2	0.394	0.505	4.30	1.39
	3	0.402	0.512	4.41	1.41

长上下文架构已将 token 窗口扩展到几十万甚至数百万，我们的实验表明，当模型接触结构不良或语义稀释的历史时，性能依然会下降。这与关于上下文“腐化”的新兴经验证据一致：当 Transformer 在冗长、异构的序列上分配注意力时，其性能会恶化。PAACE 表明，显式建模计划结构和未来 k 步相关性会带来显著收益。查询感知压缩方法为单个即将到来的问题优化，而 PAACE 会保留多步骤链所需的信息，例如工具调用序列、文档引用和因果依赖。PAACE 塑形后的上下文可充当连贯的状态表示，避免丢失经常损害基线方法的全局任务结构。

PAACE 的一个显著贡献是 PAACE-Syn 所生成合成工作流的有效性。现有智能体研究数据集要么没有计划标注，要么规模过小。相比之下，PAACE-Syn 会生成数百万条多样化轨迹，其中具有显式计划结构、多跳依赖、可变噪声和丰富的工具使用模式。这些工作流为训练未来 k 步感知压缩器提供密集监督，而无须昂贵的人工标注。蒸馏后的 PAACE-FT 能恢复教师行为的至多 97%，其强劲性能表明，合成监督能够可靠地迁移到紧凑模型。这一结果对 PAACE 的实际部署意义重大：学生压缩器可以在智能体循环的每一步低成本执行，从而摆脱对大型教师 LLM 的依赖。

另一个有趣的观察是，PAACE 经常超过“无压缩”基线。这说明压缩不只是提高效率的工具，也是一种正则化。

移除无关或矛盾的信息，会迫使智能体在更连贯、更有结构的状态上进行推理，从而减少干扰，并降低关注伪相关细节的风险。这或许可以解释 OfficeBench 和 8-Objective QA 上更高的任务成功率，因为其中未经筛选的历史包含冗长日志和重复工具输出。我们的结果表明，PAACE 显著缓解了指令漂移，这是长时间运行工作流中的一种常见故障模式。通过以未来 k 个任务作为压缩条件，PAACE 保持了智能体计划与演化状态之间的一致。这保证了重写后的指令、保留的约束和上下文细节在各步骤间保持一致，减少级联错误的累积。

5 结论

本文提出 PAACE，一个面向长时程 LLM 智能体的计划感知自动化上下文工程框架。PAACE 将上下文建模为结构化、依赖计划的状态，并通过相关性评分、重写、摘要、裁剪和指令协同精炼，利用结果层监督对其进行优化，使智能体能够在严格的上下文预算下开展更稳健的推理。在多个基准上，PAACE 在大幅降低峰值上下文长度的同时，持续改善任务正确性、推理稳定性和有效上下文利用率。这些结果表明，学习式、计划感知的上下文塑形可以像知识密集型系统中的检索增强一样，成为智能体架构的一等组件。

PAACE 的目标是稳健性和上下文效率，而不是保证降低 API 总成本；后者取决于部署方式和压缩频率。我们不声称方法具有收敛性、最优性或形式化保证；相反，我们的结果说明，在真实多步骤工作流中，由结果驱动的学习足以诱导出稳健且可迁移的压缩行为。PAACE 有意学习环境专用与工作流专用的压缩策略。跨领域泛化虽是未来工作的重要方向，但我们的发现表明，在长时程设定中，计划感知的相关性本质上依赖任务；这种专用性是有效上下文工程的特征，而非局限。

总体而言，PAACE 将上下文工程表述为一个可学习的计划感知优化问题，并为构建能够在长时间交互中连贯运行的可扩展、可靠智能体提供了实用的经验基础。

更广泛影响声明

从更广泛的影响看，提高上下文效率可以使长时程智能体在更严格的资源、延迟与成本约束下部署，从而有可能扩大复杂智能体系统在实际环境中的可及性。同时，如果在超出训练条件的环境中使用，激进或未经充分验证的压缩策略可能会遗漏与安全相关的指令、约束或来源信息。PAACE 通过要求压缩结果与完整上下文执行在结果层面一致，并丢弃降低任务成功率的压缩来缓解这一风险。尽管如此，PAACE 并不是安全或对齐机制；若要用于安全关键或高风险领域，还需要额外验证、领域专用检查或人工监督。

参考文献

- Ahn, M. et al. Mrkl systems: Modular reasoning, knowledge and language. *arXiv preprint arXiv:2205.00445*, 2022.
- Asai, A., Chen, X., and Hajishirzi, H. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2403.14580*, 2024.
- Chung, H. W., Hou, L., Longpre, S., et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.
- Gao, L., Madaan, A., et al. Pal: Program-aided language models. In *ICML*, 2023.
- Guo, M. et al. Longt5: Efficient text-to-text transformer for long sequences. In *arXiv preprint arXiv:2112.07916*, 2021.
- Huang, J. et al. Planning with large language models via iterative refinement. In *NeurIPS*, 2024.
- Jiang, H., Ye, F., et al. LlmLingua: Efficient context compression for large language models. *arXiv preprint arXiv:2310.07041*, 2023.
- Jiang, H. et al. LlmLingua-2: Efficient and robust context compression for large language models. *arXiv preprint arXiv:2404.17762*, 2024.
- Joo, T., Ishida, S., Sosnovik, I., Lim, B., Rezaei-Shoshtari, S., Gaier, A., and Giaquinto, R. Graph of agents: Principled long-context modeling by emergent multi-agent collaboration. *arXiv preprint arXiv:2509.21848*, 2025. URL <https://arxiv.org/abs/2509.21848>.
- Lewis, M. et al. Bart: Denoising sequence-to-sequence pre-training. In *ACL*, 2019.
- Li, J., Zhou, X., Zhang, Y., Zhao, R., and Chen, Y. Compass: Controllable memory policy for long-context language models. *arXiv preprint arXiv:2402.05618*, 2024.
- Liu, X., Guo, J., et al. Agentbench: Evaluating llm agents via simulated environments. In *NeurIPS Datasets and Benchmarks Track*, 2023. URL <https://arxiv.org/abs/2308.03688>.
- Ouyang, L. et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.
- Park, J. et al. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- Pryzant, R. et al. Automated prompt engineering. In *NeurIPS*, 2023.
- Rae, J. et al. Compressive transformers for long-range sequence modeling. *ICLR*, 2020.
- Rafailov, R. et al. Direct preference optimization: Optimizing language models without reinforcement learning. *arXiv preprint arXiv:2305.18290*, 2023.
- Raffel, C. et al. Exploring the limits of transfer learning with a unified text-to-text transformer. In *Journal of Machine Learning Research*, 2020.
- Saunders, W. et al. Preference ranking optimization. *arXiv preprint arXiv:2402.07872*, 2024.
- Schick, T., Dwivedi-Yu, J., et al. Toolformer: Language models can teach themselves to use tools. In *International Conference on Learning Representations (ICLR)*, 2023a. URL <https://arxiv.org/abs/2302.04761>.
- Schick, T. et al. Toolformer: Language models can teach themselves to use tools. In *ICLR*, 2023b.
- See, A. et al. Get to the point: Summarization with pointer-generator networks. *ACL*, 2017.
- Shinn, N. et al. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- Sun, S., Liu, Z., Chen, X., and Li, J. Prompt compression for large language models. *arXiv preprint arXiv:2310.06201*, 2023.
- Sun, W., Lu, M., Ling, Z., Liu, K., Yao, X., Yang, Y., and Chen, J. Scaling long-horizon llm agents via context-folding. *arXiv preprint arXiv:2510.11967*, 2025. URL <https://arxiv.org/abs/2510.11967>.
- Trivedi, H., Khot, T., Hartmann, M., Manku, R., Dong, V., Li, E., Gupta, S., Sabharwal, A., and Balasubramanian, N. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL <https://doi.org/10.18653/v1/2024.acl-long.850>.
- Wang, Z., Cui, Y., Zhong, L., Zhang, Z., Yin, D., Lin, B. Y., and Shang, J. Officebench: Benchmarking language agents across multiple applications for office automation. *arXiv*

- preprint *arXiv:2407.19056*, 2024. URL <https://doi.org/10.48550/arXiv.2407.19056>.
- Wei, J. et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.
- Wu, Y. et al. Provence: Learned context pruning for retrieval-augmented generation. *arXiv preprint arXiv:2403.16975*, 2024.
- Yao, S., Bosma, D., Zhao, J., et al. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023a. URL <https://arxiv.org/abs/2210.03629>.
- Yao, S. et al. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023b.
- Yao, S. et al. Tree of thoughts: Deliberate problem solving with large language models. In *arXiv preprint arXiv:2305.10601*, 2023c.
- Yu, B. et al. Memagent: Learned memory mechanisms for long-context language models. *arXiv preprint arXiv:2501.01234*, 2025a.
- Yu, Y., Li, H., Zheng, J., et al. Agent context optimization: Compressing agent interaction histories via natural language guidance. *arXiv preprint arXiv:2501.00001*, 2025b.
- Zaheer, M. et al. Big bird: Transformers for longer sequences. *NeurIPS*, 2020.
- Zhang, J. et al. Pegasus: Pre-training with extracted gap sentences for abstractive summarization. *ICML*, 2019.
- Zhang, T. et al. React in the wild: Reasoning-augmented tool use. *arXiv preprint arXiv:2306.03631*, 2023.
- Zhang, Z. et al. Recap: Recursive context-aware reasoning and planning for large language model agents. *arXiv preprint arXiv:2510.23822*, 2025. URL <https://arxiv.org/abs/2510.23822>.
- Zhou, J. et al. Large language model prompt optimization. *arXiv preprint arXiv:2309.03409*, 2023a.
- Zhou, S. et al. Large language models as tool makers. *arXiv preprint arXiv:2304.03439*, 2023b.
- Zhou, Y. et al. Recursive chain of thought: Multi-step reasoning with structured steps. *arXiv preprint arXiv:2311.11543*, 2023c.
- Zhou, Z., Zhang, Z., et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2307.15238>.
- Zhou, Z., Qu, A., Wu, Z., Kim, S., Prakash, A., Rus, D., Zhao, J., Low, B. K. H., and Liang, P. P. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*, 2025. URL <https://doi.org/10.48550/arXiv.2506.15841>.